Recycling Executed Branch Outcomes to Improve Branch Prediction

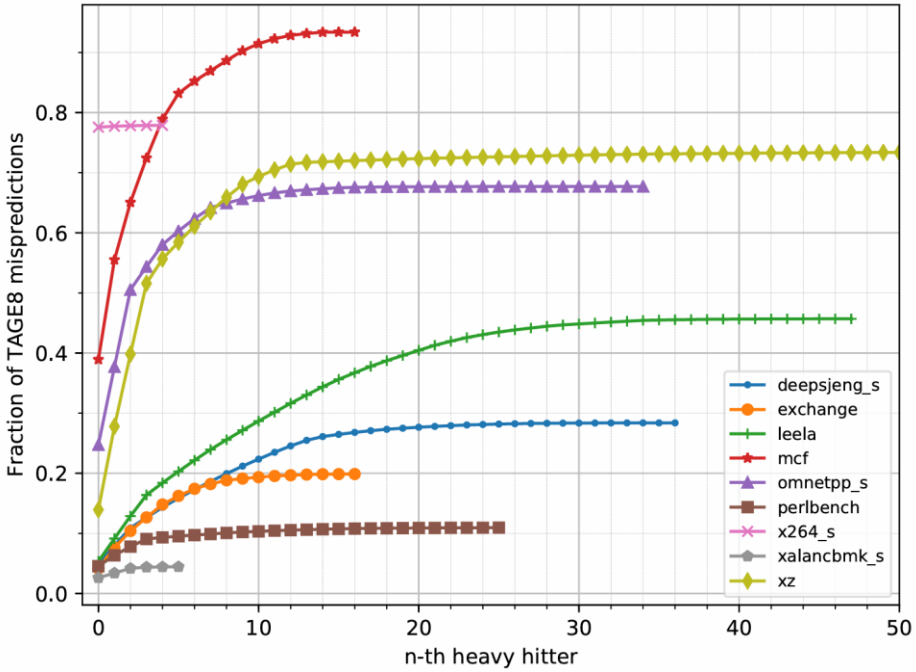
Neel Mandal
Advisor: David Schall
Systems Research Group
https://dse.in.tum.de/

20.10.2025 – 20.02.2026

Introduction

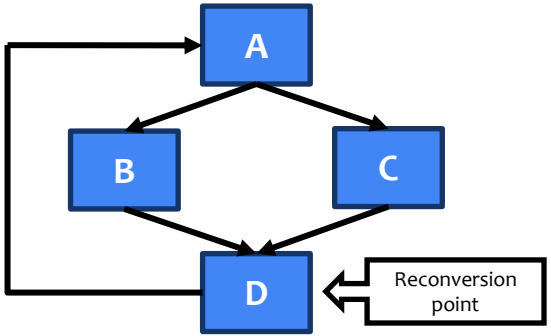
- Issue: Mispredictions hurt CPU performance
  - Up to hundreds of instructions may be wasted

    → Energy waste
- The main performance hit is caused by a few hard-to-predict branches that are data-driven and not history-driven [1]
- Executed information is lost during squash/rollback



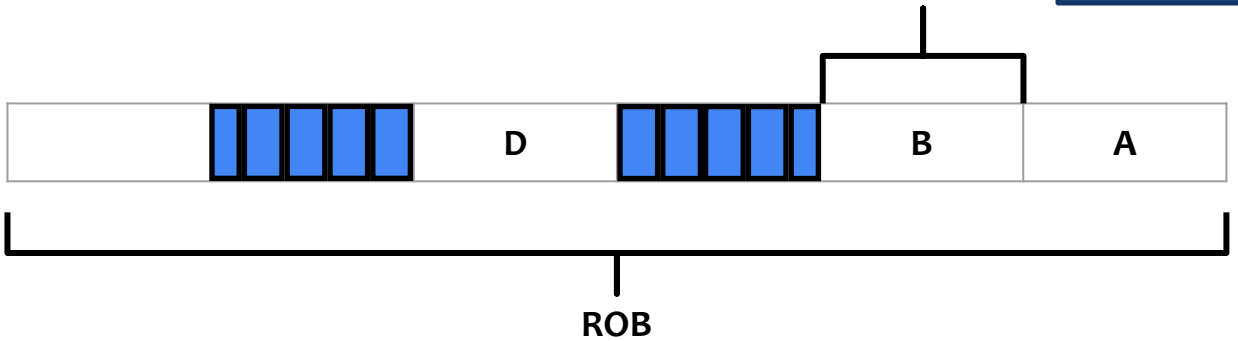
Source: [1], Fig. 2.

Research Gap

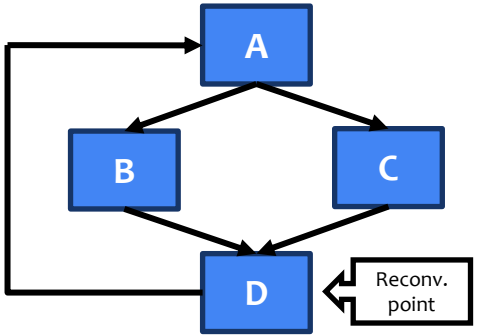


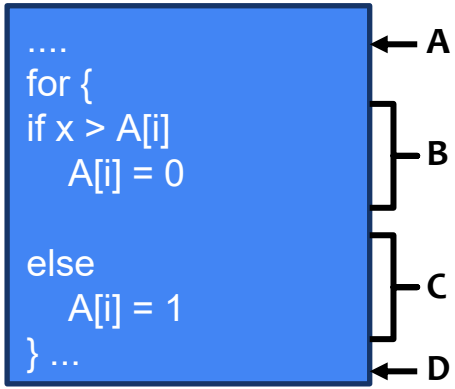

```
....
for {
if x > A[i]
    A[i] = 0

else
    A[i] = 1
} ...
```

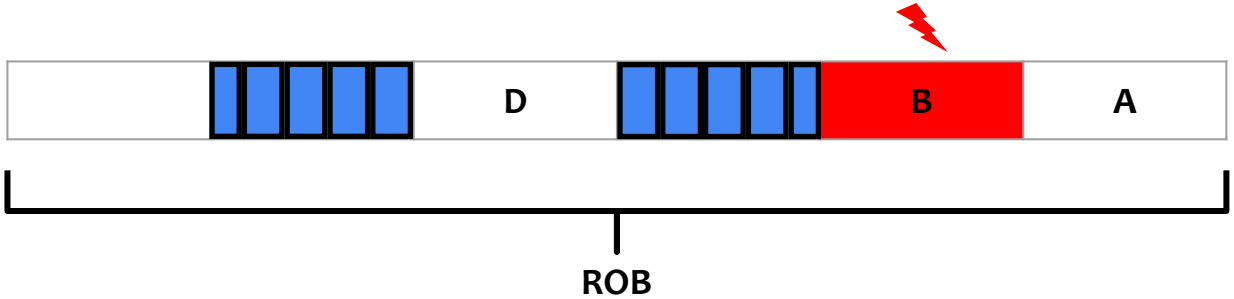
A
B
C
D



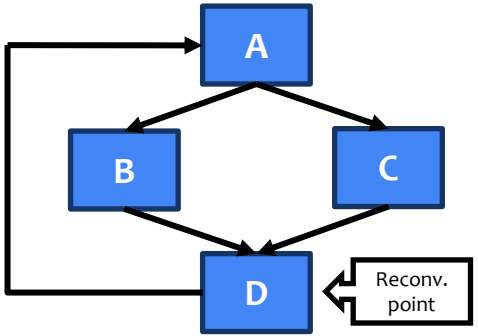
Research Gap





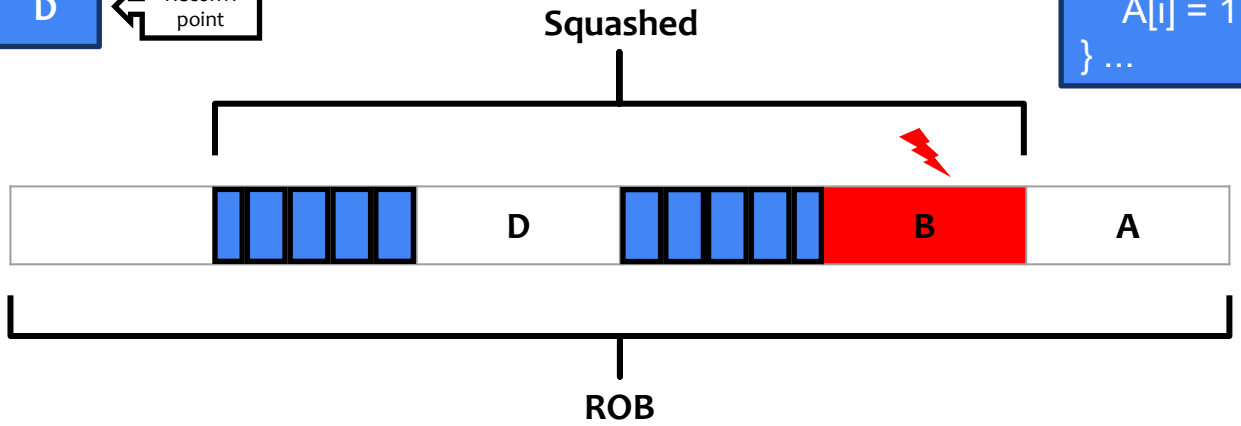


Research Gap

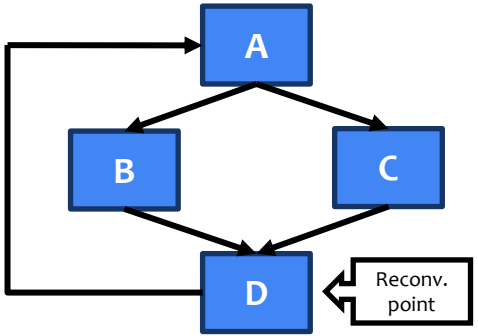


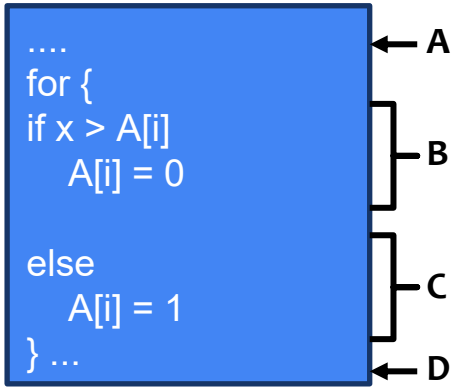

```
....
for {
if x > A[i]
    A[i] = 0

else
    A[i] = 1
} ...
```

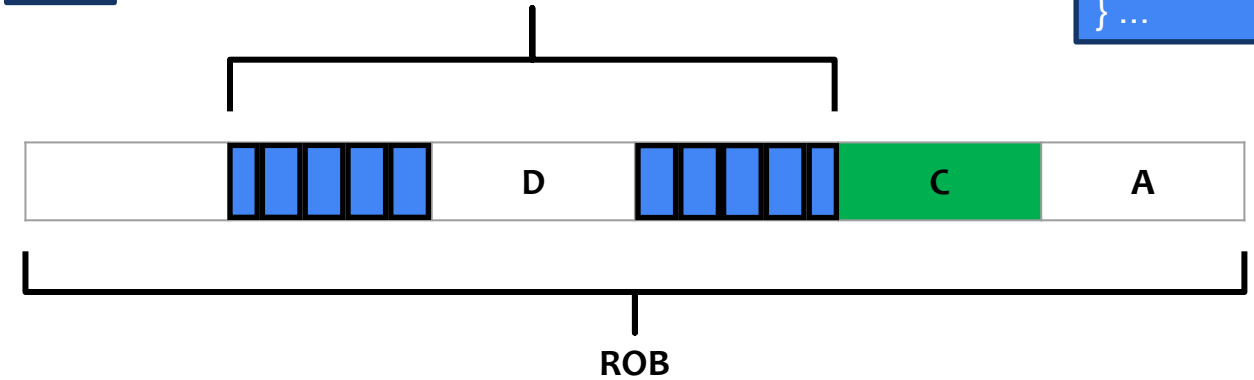
A
B
C
D



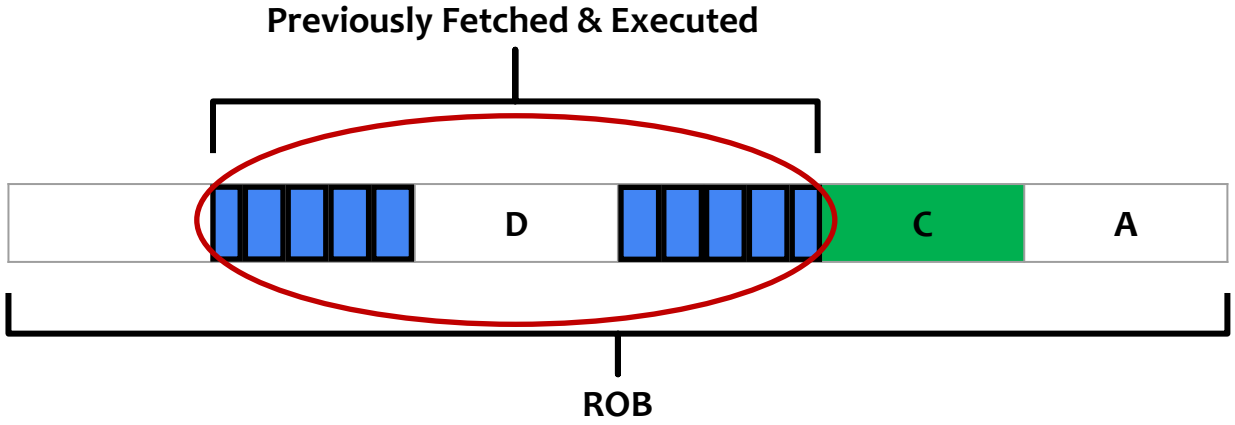
Research Gap







Research Gap



Can previous false path branch outcomes be reused?

Research Goal

- Develop and test the functional ability of branch recycling
- Analyze storage needs of branch recycling
- Does branch recycling reduce the impact of systematically hard-to-predict branches?

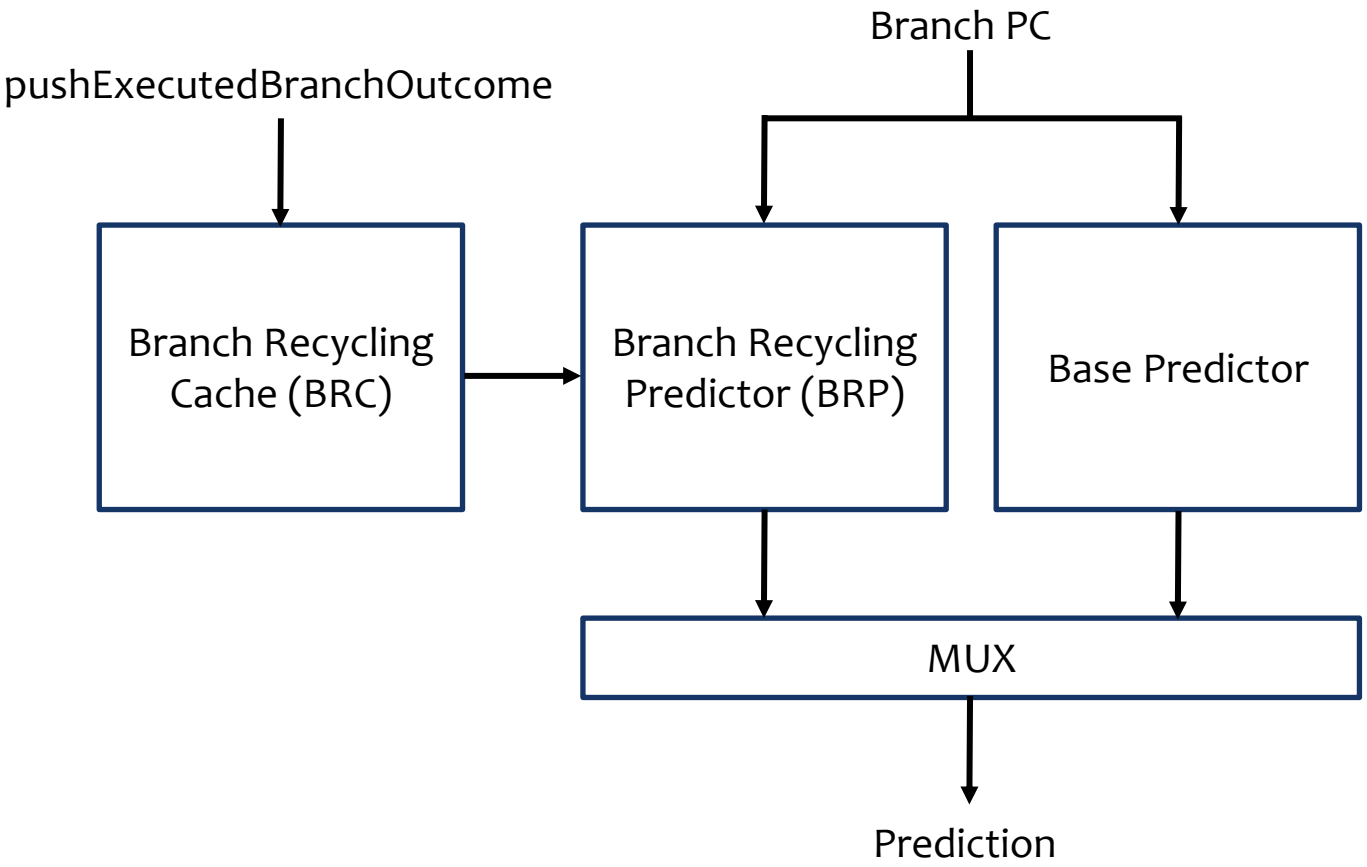
Structure

1. Introduction - Motivation
2. Contributions
3. System Overview – Design
4. Evaluation
5. Summary and Conclusion
6. Future Work

Contributions

- Design and implementation of a branch recycling predictor (BRP) in gem5
- Self-managed data structure (BRC) for storing executed branch outcomes
- Two control mechanisms for recycling
  - Training
  - Stride Detection
- Evaluation of BRP with a custom microbenchmark and standard test suite

System Overview - Design



System Overview – Design: BRC

Bucket

Branch Recycling Cache (BRC)					
Branch	LFU-Counter	Training Metric	Execs	Use Recycle	FIFO
0x01	5	5	3	TRUE	[0; 1; 0]
0x02	3	4	4	FALSE	[]
0x03	5	7	2	TRUE	[]
0x04	8	9	2	TRUE	[1; 1; 1]
0x05	0	0	0	FALSE	[]

pushExecutedOutcome

System Overview – Design: BRC Eviction

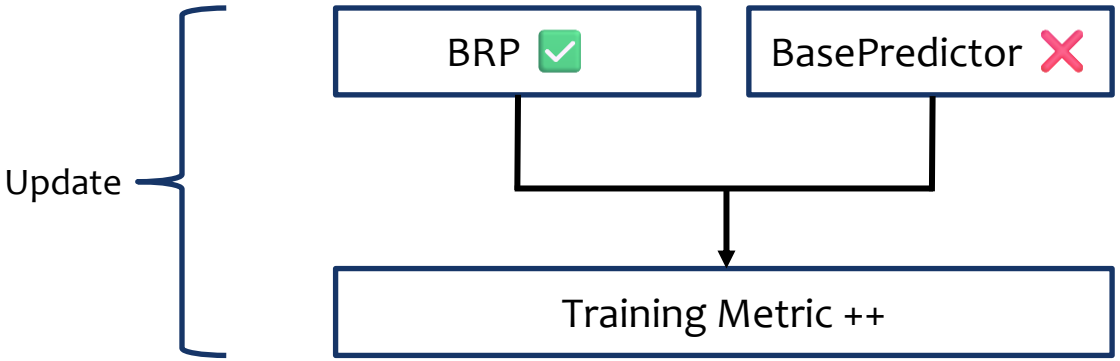
Type	Eviction Policy
Buckets	LFU – Age based tie breaker
FIFO Queues	Oldest Entry

Simple policies to cap storage

System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	False

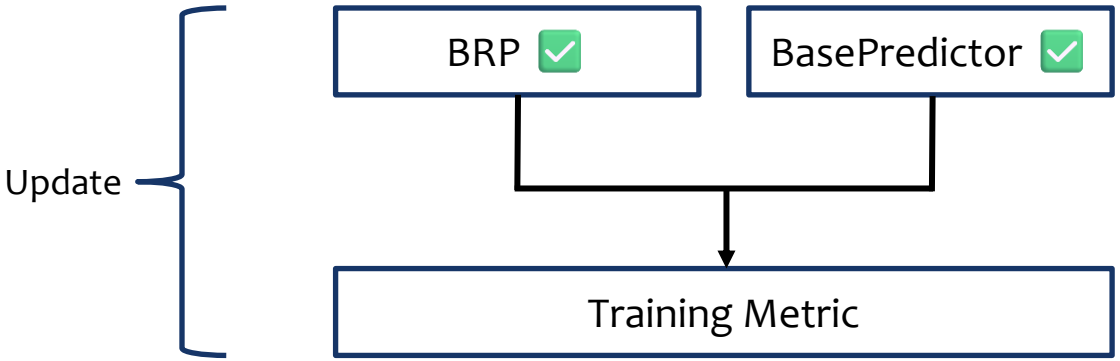
Execs	Training Metric
0	0



System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	False

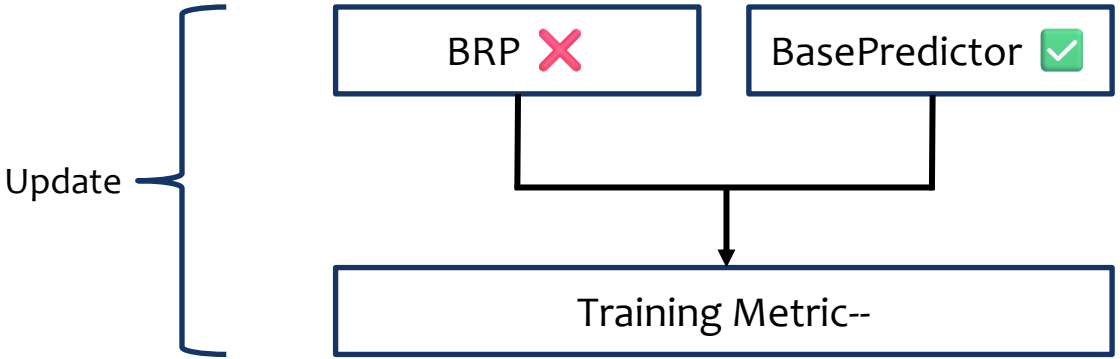
Execs	Training Metric
1	1



System Overview – Design: Control Mechanisms

Training Interval	Use Recycling
5	False

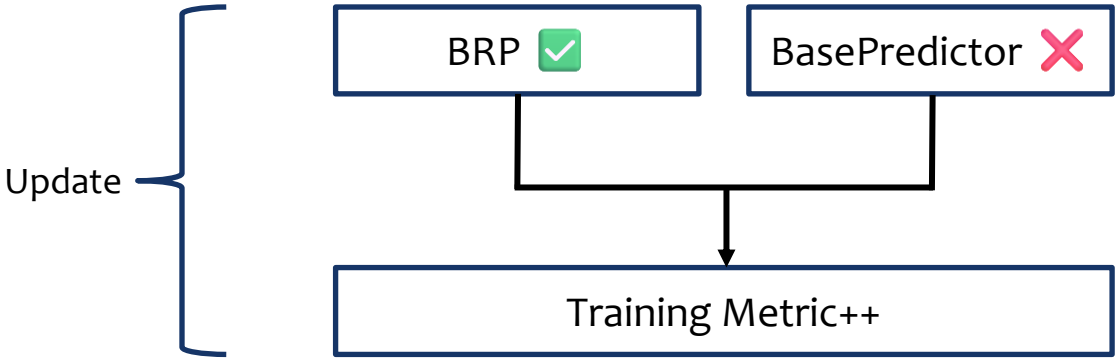
Execs	Training Metric
2	0



System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	False

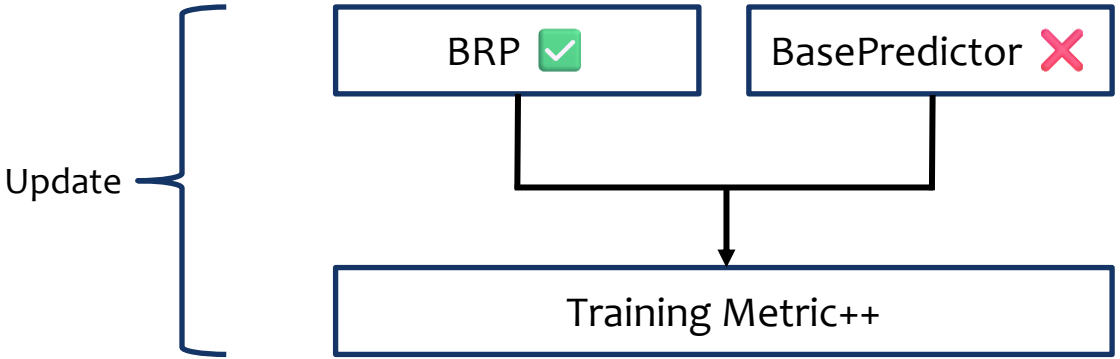
Execs	Training Metric
3	1



System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	False

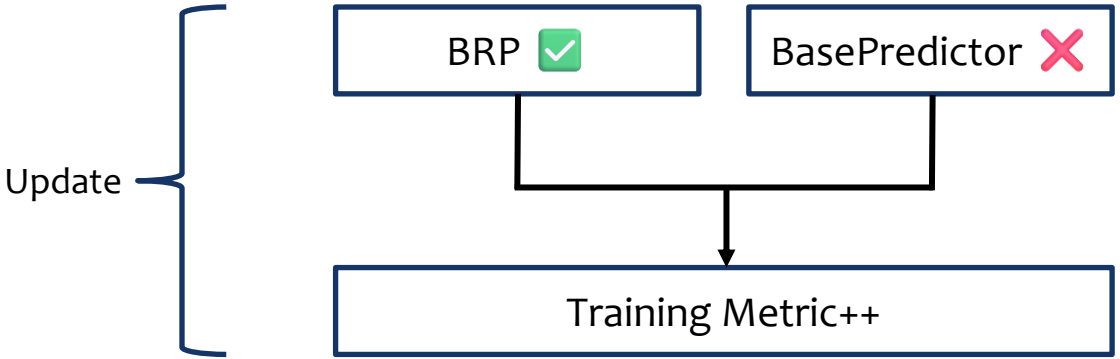
Execs	Training Metric
4	2



System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	True

Execs	Training Metric
5	3



System Overview – Design: Control Mechanisms Training

Training Interval	Use Recycling
5	True

Execs	Training Metric
0	0

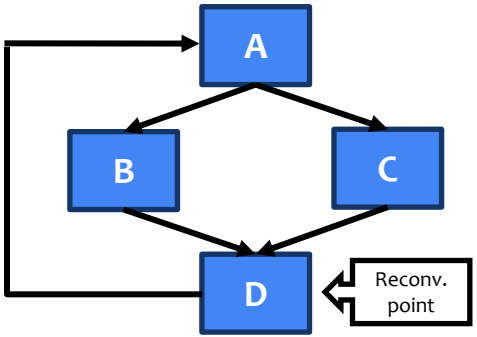
Update

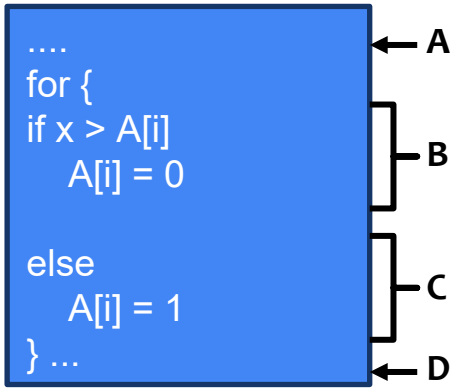
```
if (Training Metric > 0)
    Use Recycling = True
else
    Use Recycling = False

Execs = 0
Training Metric = 0
```

Training Metric > 0 ⇒ Use Recycling = True

System Overview – Design: Idea Stride

i	A[i]
0	5
1	9
2	6
3	10
4	4
5	3
6	1
7	8
8	2
9	7



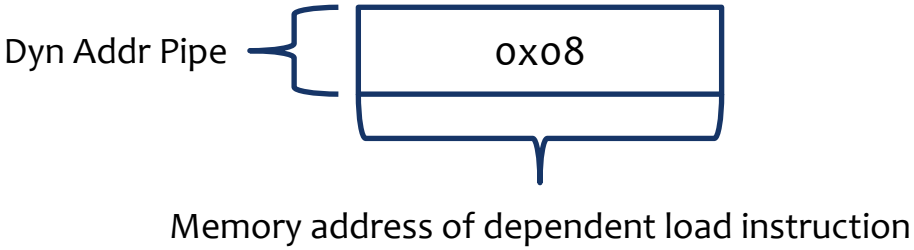


Utilize dynamic memory access patterns

System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

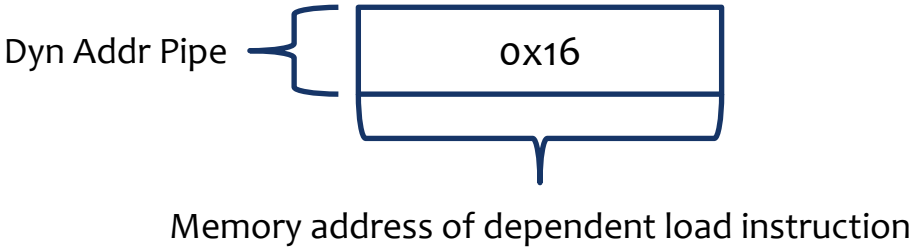
Stride Confidence	Last Dyn Addr	Stride
0	0	0



System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

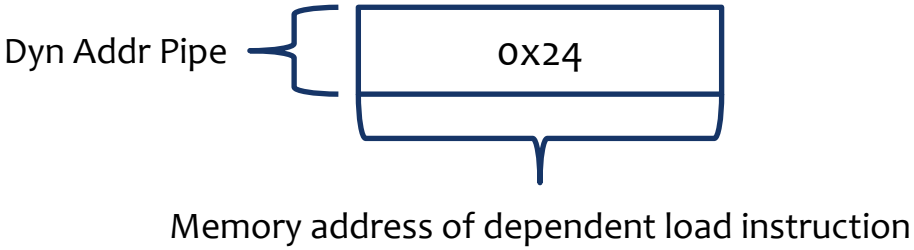
Stride Confidence	Last Dyn Addr	Stride
1	0x08	8



System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

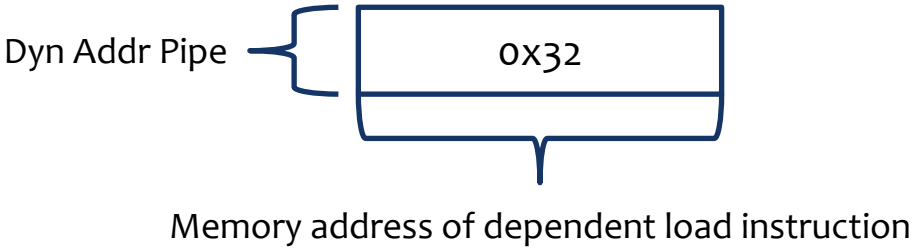
Stride Confidence	Last Dyn Addr	Stride
2	0x16	8



System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

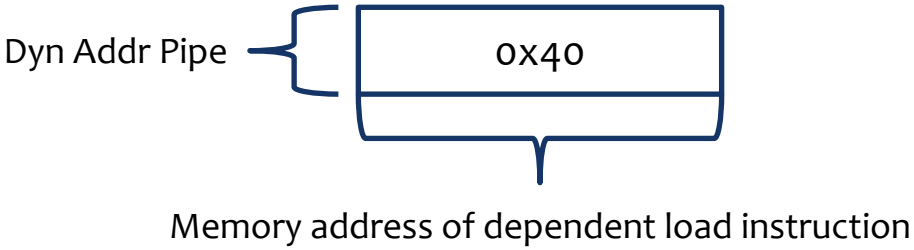
Stride Confidence	Last Dyn Addr	Stride
3	0x24	8



System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

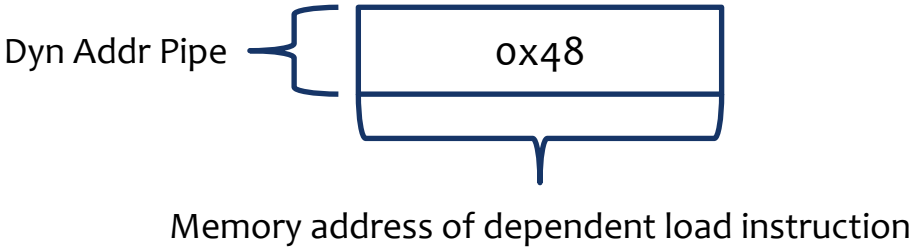
Stride Confidence	Last Dyn Addr	Stride
4	0x32	8



System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

Stride Confidence	Last Dyn Addr	Stride
5	0x40	8

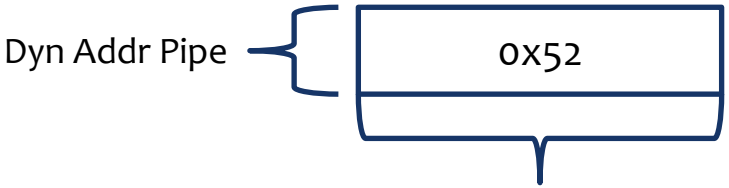


Stride has reached the Threshold of 5 ⇒ Recycling activated during lookup

System Overview – Design: Control Mechanisms Stride Detection

Stride Threshold	5

Stride Confidence	Last Dyn Addr	Stride
0	0x48	8



Memory address of dependent load instruction

Current Stride (4) does not match last Stride (8) ⇒ Recycling deactivated and counter reset

Evaluation: Experimental Setup

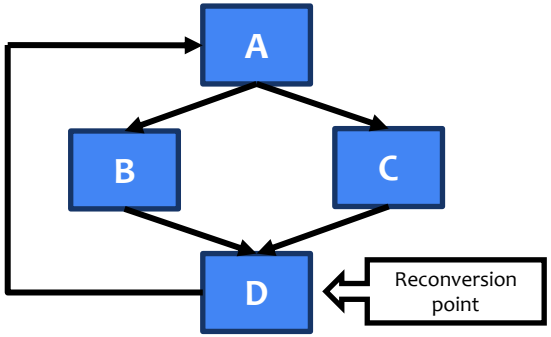
Component	Configuration
Simulator	gem5 v. 23.0.1.0
Architecture	gem5 ARM ISA – full system mode
Processor Model	out-of-order (O3) CPU model | Intel Sapphire Rapids
Clock Frequency	3 GHz
Core Configuration	SPEC workloads: 1 core | Server workloads: 2 cores
Core Microarchitecture	6-way OoO, 512 ROB, 248/122 LQ/SQ
Memory	3 GiB dual-channel DDR4-2400
Branch Predictor	TAGE-SC-L (64 kB) and BRP with training and stride detection
BTB	16K entries, 8-way associative
Cache	32 KiB 8-way L1-I, 64 KiB 16-way L1-D, 2 MiB 16-way L2
Prefetcher	Instructions: FDIP + Tagged; Data: Stride

Evaluation: Simulator Parameters

Parameter	Description
Enable Recycling	Enable recycling to support base predictor
Enable Training	Enable the training control mechanism for recycling
Enable Stride	Enable the stride control mechanism for recycling
Bucket Count	Set the bucket count of the BRC
FIFO Size	Set the size of the per bucket FIFO queues
Training Interval	Set number of executions per training window
Stride Threshold	Set stride confidence threshold to activate recycling

Evaluation: Methodology

Evaluation: Microbenchmark




```
....
for {
if x > A[i]
    A[i] = 0

else
    A[i] = 1
} ...
```

A
B
C
D

Evaluation: Microbenchmark

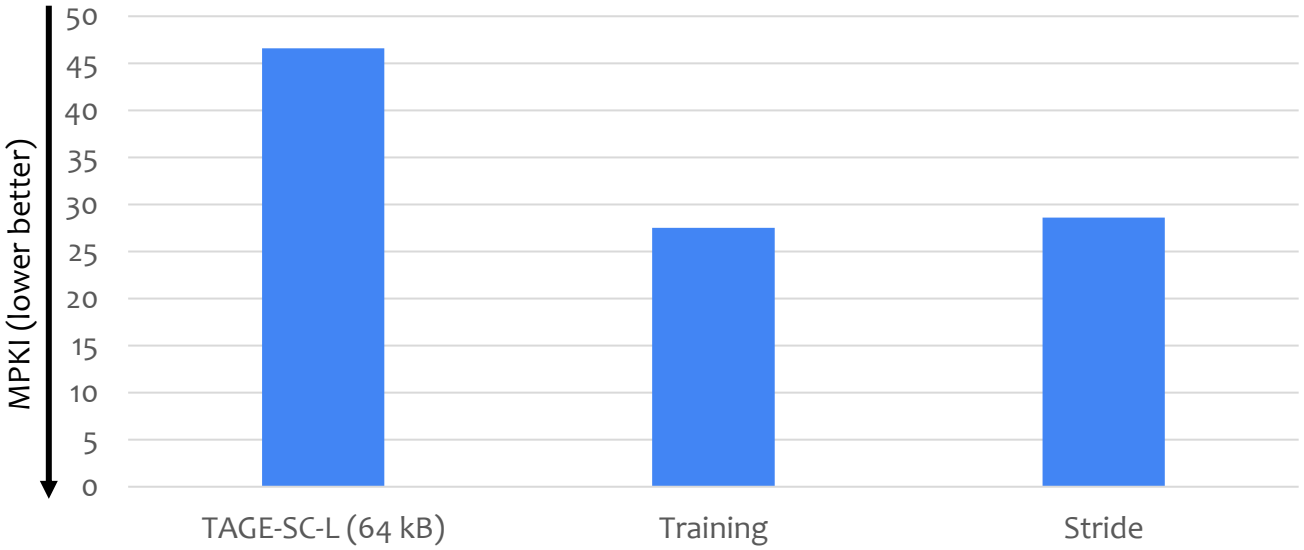


Table: Reduction in mispredictions relative to TAGE-SC-L in % (higher better)

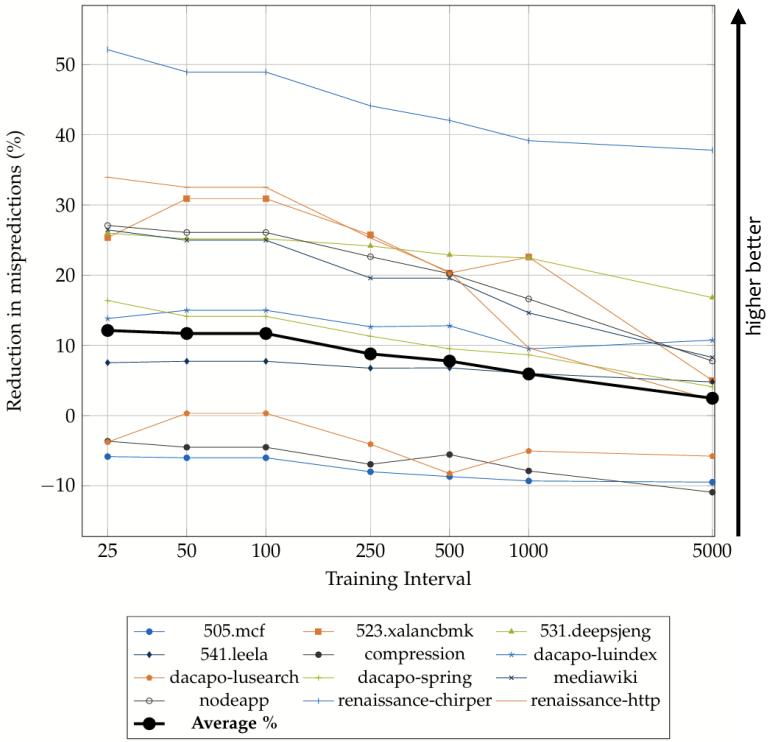
Training	Stride
40.94%	38.57%

Recycling reduces MPKI by 40.94%

Evaluation: Benchmark Suite

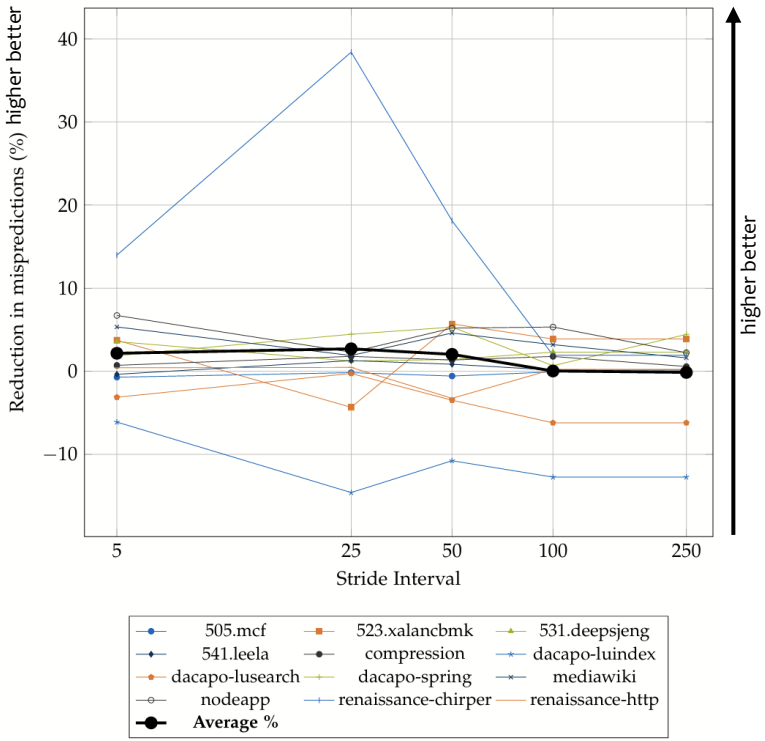
Benchmark	Language	Description	MPKI TAGE-SC-L
505.mcf	C	single-depot vehicle sched. in pub. transp.	24.91
523.xalancbmk	C++	XSL Transformation	2.37
531.deepsjeng	C++	AI - alpha-beta tree search & pattern recog.	9.85
541.leela	C++	AI - Monte Carlo simulation for GO	14.35
compression	C++	Snappy, ZSTD, Brotli, and Zlib	14.71
dacapo-luindex	Java	Text indexing tool	2.97
dacapo-lusearch	Java	Text search tool	6.54
dacapo-spring	Java	Spring based application	6.39
mediawiki	PHP	Mediawiki page	9.32
nodeapp	JavaScript	Small shopping website	6.99
renaissance-chirper	Scala/Java	Microblog. service using X Finagle	3.21
renaissance-http	Scala/Java	Sends small Finagle req. to HTTP server	2.99
Average MPKI	-	-	**9.53**

Evaluation: Benchmark Suite – Training Interval



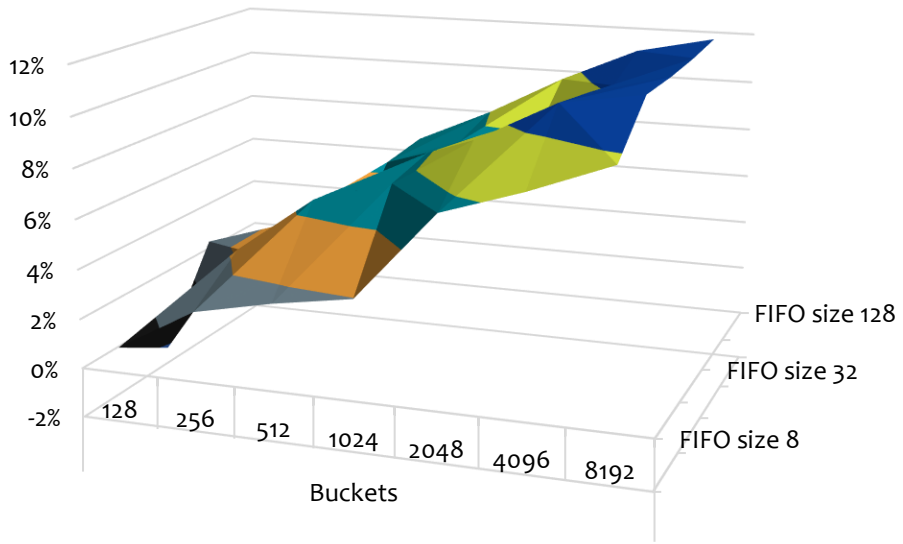
Training Interval of 5 reduces mispredictions by 12.78% relative to TAGE-SC-L

Evaluation: Benchmark Suite – Stride Threshold



Stride Threshold of 25 reduces mispredictions by 2.73% relative to TAGE-SC-L

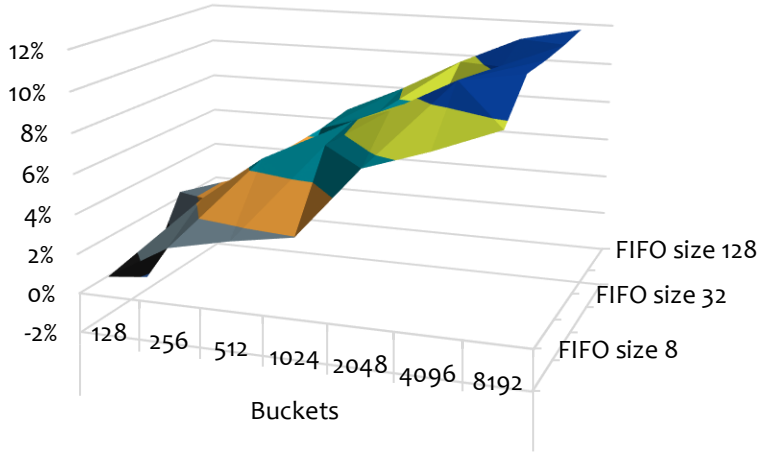
Evaluation: BRC Capacity – Training



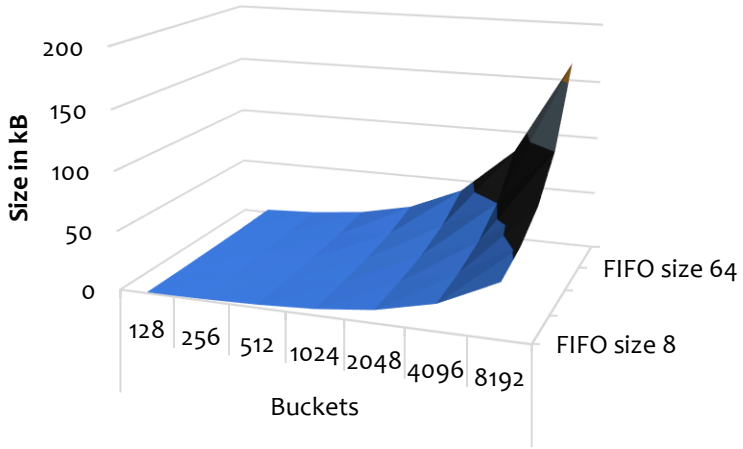
Average mispredictions relative to TAGE-SC-L in % (higher better)

Bucket count is the driving factor for performance

Evaluation: BRC Capacity – Training



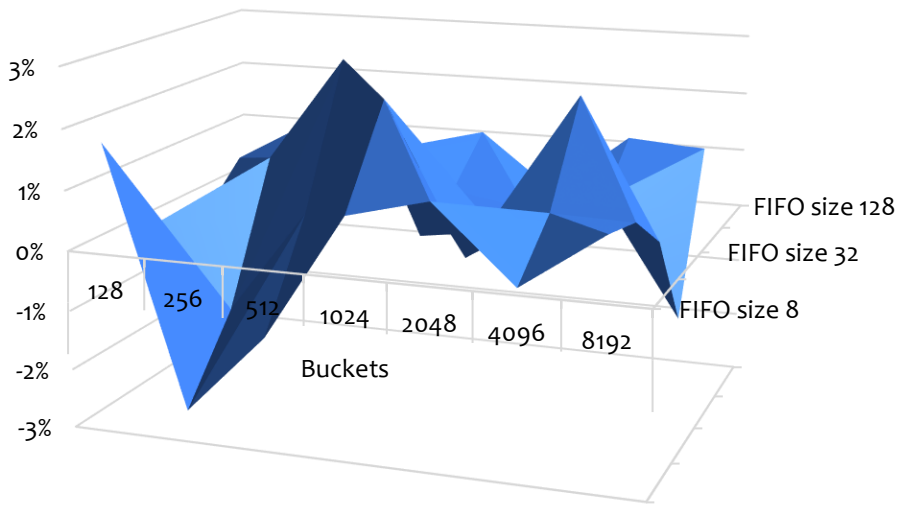
Average mispredictions relative to TAGE-SC-L in % (higher better)



BRC storage needs in kB by Buckets and FIFO size

Best balance achieved with 4096 Buckets, FIFO size 16 (26 kB)
⇒ 10.97% reduction in mispredictions relative to TAGE-SC-L
⇒ 2.12% increase in mispredictions relative to uncapped BRC

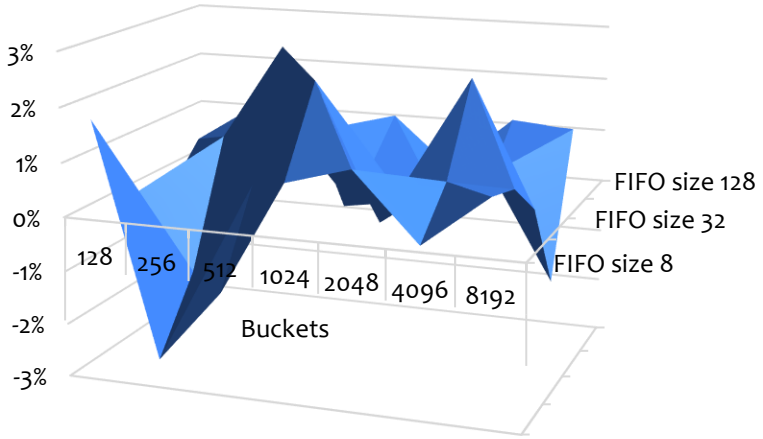
Evaluation: BRC Capacity – Stride



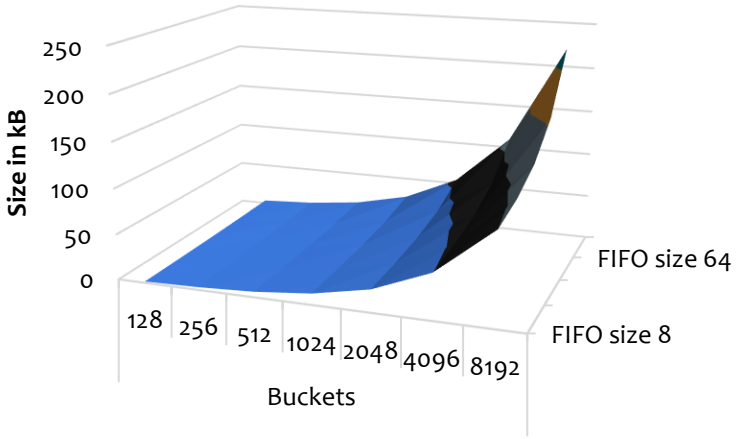
Average mispredictions relative to TAGE-SC-L in % (higher better)

No clear driving factor visible

Evaluation: BRC Capacity – Training



Average mispredictions relative to TAGE-SC-L in % (higher better)



BRC storage needs in kB by Buckets and FIFO size

Best balance achieved with 512 Buckets, FIFO size 32 (6.5 kB)
⇒ 2.78% reduction in mispredictions relative to TAGE-SC-L
⇒ 0.81% reduction in mispredictions relative to uncapped BRC

Evaluation: Summary Control Mechanisms

	Training	Stride Detection
Best Balance Config	10.97% (26 kB)	2.78% (6.5 kB)
Similar Storage	7.43% (6.25 kB)	2.78% (6.5 kB)

Reduction in mispredictions relative to TAGE-SC-L in % (higher better)

Training is the superior control mechanism even under equal storage constraints

Evaluation: HTP-Branch Analysis Precision Control Mechanisms

- HTP branch: Branches that contribute to 90% of mispredictions under TAGE-SC-L
- How many HTP branches benefit from recycling with either training or stride detection

Benchmark	Branch Count	HTP Branch Count	Training	Stride Detection
505.mcf	104	10	30.00%	40.00%
531.deepsjeng	889	206	83.98%	60.98%
…				
AVERAGE			**62%**	**49.13%**

Both mechanisms achieve a low rate in identifying beneficial branches

Evaluation: HTP-Branch Analysis Potential

- Potential: For each branch, the mispredictions of either the base predictor or recycling are accumulated based on which predictor performs better.
- Analyze room for improvement with better predictor choice

Benchmark	Training	Potential Training	Stride	Potential Stride
505.mcf	-5.51%	0.80%	-0.31%	0.63%
…				
nodeapp	22.18%	27.25%	13.81%	20.48%
AVERAGE	**13.48%**	**19.88%**	**1.17%**	**8.02%**

Mispredictions relative to TAGE-SC-L of HTP - branches in % (higher better)

A perfect predictor choice eliminates the negative impact of recycling across all workloads

Summary and Conclusion

- Design & Implementation
  - Evaluated recycling executed branch outcomes for improved branch prediction
  - Recycles resolved false-path outcomes to support a base predictor
  - Implemented BRP with recycling in gem5 to support TAGE-SC-L
  - Implementation of bounded BRC to limit overhead
  - Implementation of control mechanisms:
    - Per-branch training
    - Stride detection
- Results:
  - Microbenchmark: > 40% misprediction reduction
  - Benchmark Suite: Up to 12.8% average misprediction reduction
  - Mitigates the impact of HTP branches but does not fully solve it

Effectiveness and Low overhead make the BRP a promising complementary technique for future branch predictors

Future Work

- BRC: Limit the global number of stored outcomes in the BRC
- Training Mechanism: Develop more advanced training strategies to better identify beneficial branches
- Resource Usage: Evaluate hardware implementation & energy consumption

Backup

Abbreviations

- BRC: Branch Recycling Cache
- BRP: Branch Recycling Predictor
- HTP: Hard-to-Predict branches (branches that contribute to 90% of mispredictions)
- LFU: Least-Frequently Used
- MPKI: Mispredictions per Kilo Instructions

Evaluation: BRC Capacity – Bucket Count Training

Buckets	256	512	1024	2048	4096	8192
505.mcf	0,00%	0,00%	-2,53%	-2,53%	-2,53%	-2,53%
523.xalancbmk	1,49%	-3,10%	-9,69%	-0,10%	-0,10%	-0,10%
531.deepsjeng	9,92%	2,51%	-0,06%	-0,06%	-0,06%	-0,06%
541.leela	1,06%	0,44%	0,02%	0,02%	0,02%	0,02%
compression	-0,52%	0,00%	0,00%	0,00%	0,00%	0,00%
dacapo-luindex	7,89%	0,96%	0,00%	0,55%	-0,46%	0,71%
dacapo-lusearch	0,42%	0,03%	1,26%	-0,68%	-0,28%	-0,42%
dacapo-spring	15,84%	15,12%	15,29%	6,35%	3,24%	1,44%
mediawiki	23,65%	21,52%	14,64%	11,54%	6,53%	5,48%
nodeapp	28,32%	24,58%	23,20%	17,42%	13,86%	11,25%
renaissance-chirper	47,90%	47,80%	15,08%	17,84%	7,51%	11,10%
renaissance-http	31,20%	27,10%	17,76%	19,95%	10,94%	5,27%
AVERAGE	**10,22%**	**8,60%**	**5,30%**	**4,03%**	**2,34%**	**1,88%**

Mispredictions relative to uncapped BRC by bucket count in % (lower better)

Increase in Buckets leads to fewer mispredictions

Evaluation: BRC Capacity – Bucket Count Stride

Buckets	256	512	1024	2048	4096	8192
505.mcf	0,02%	0,02%	0,02%	0,02%	0,02%	0,02%
523.xalancbmk	-11,83%	-10,59%	0,82%	-0,19%	-0,19%	-0,19%
531.deepsjeng	2,74%	-2,30%	0,01%	0,01%	0,01%	0,01%
541.leela	1,07%	0,50%	-0,03%	-0,03%	-0,03%	-0,03%
compression	0,21%	-0,01%	-0,01%	-0,01%	-0,01%	-0,01%
dacapo-luindex	-5,44%	-0,61%	4,66%	-2,87%	2,44%	-0,01%
dacapo-lusearch	-3,24%	2,00%	-0,33%	-3,71%	-4,82%	-1,13%
dacapo-spring	2,14%	3,75%	1,08%	5,32%	2,73%	1,72%
mediawiki	4,43%	0,13%	0,28%	2,61%	-1,84%	-2,18%
nodeapp	-0,44%	0,46%	-1,41%	-2,18%	-2,83%	-0,91%
renaissance-chirper	32,85%	-2,22%	46,13%	31,75%	31,95%	42,65%
renaissance-http	0,95%	2,62%	3,40%	3,83%	3,48%	3,17%
AVERAGE	**2,06%**	**0,08%**	**3,02%**	**1,83%**	**1,25%**	**2,28%**

Mispredictions relative to uncapped BRC by bucket count in % (lower better)

No stable behavior with change in bucket count

Evaluation: BRC Capacity – FIFO Size Training

FIFO Size	8	16	32	64	128
505.mcf	-0,20%	-0,42%	0,12%	-0,06%	-0,16%
523.xalancbmk	14,24%	0,82%	-7,22%	1,72%	-5,78%
531.deepsjeng	2,58%	0,64%	1,34%	0,83%	0,33%
541.leela	-0,93%	-0,02%	-0,47%	1,91%	-0,48%
compression	-0,09%	-0,45%	-0,27%	0,16%	-0,12%
dacapo-luindex	2,61%	0,00%	-0,41%	-0,18%	-1,21%
dacapo-lusearch	-1,15%	-1,48%	-2,88%	-4,95%	-0,71%
dacapo-spring	1,55%	0,22%	-0,08%	-0,33%	0,11%
mediawiki	2,38%	1,05%	1,24%	1,49%	0,10%
nodeapp	1,26%	0,54%	0,98%	-0,14%	-0,40%
renaissance-chirper	34,38%	2,84%	25,65%	3,50%	2,73%
renaissance-http	10,06%	4,57%	2,20%	-5,12%	-3,78%
AVERAGE	**2,38%**	**0,14%**	**0,89%**	**0,11%**	**-0,31%**

Mispredictions relative to TAGE-SC-L in % (lower better)

Increase in FIFO size slightly reduces mispredictions

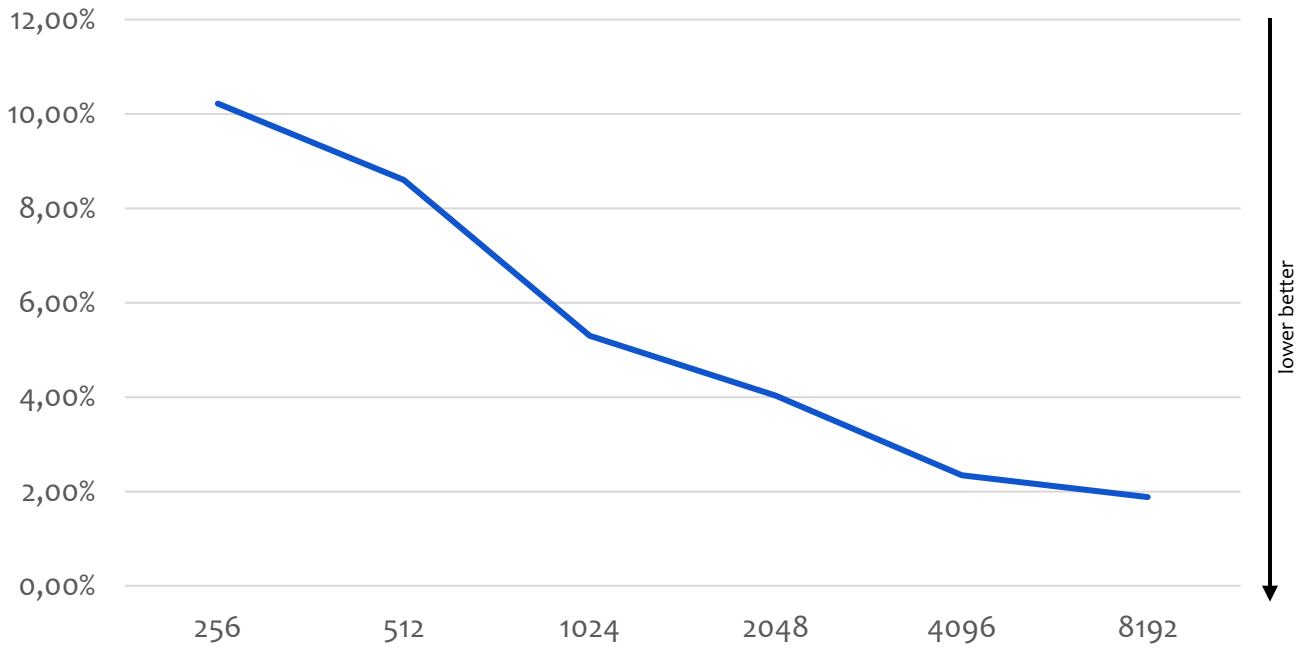
Evaluation: BRC Capacity – FIFO Size Stride

FIFO Size	8	16	32	64	128
505.mcf	0,09%	0,42%	0,06%	0,28%	0,97%
523.xalancbmk	-10,78%	-12,28%	-7,57%	3,60%	-8,99%
531.deepsjeng	-3,76%	-0,11%	-2,44%	-0,60%	1,44%
541.leela	0,36%	0,02%	0,53%	0,90%	0,06%
compression	1,54%	2,63%	1,58%	1,55%	2,58%
dacapo-luindex	-3,51%	0,33%	1,36%	-2,51%	-6,83%
dacapo-lusearch	4,30%	-5,05%	-1,74%	1,32%	-1,60%
dacapo-spring	3,49%	3,93%	1,10%	0,44%	1,83%
mediawiki	0,55%	-1,34%	-0,03%	-1,07%	2,06%
nodeapp	-1,33%	-0,21%	-4,09%	-4,80%	-2,35%
renaissance-chirper	10,81%	31,39%	47,56%	43,69%	59,39%
renaissance-http	3,51%	0,89%	1,40%	3,78%	1,44%
AVERAGE	**0,61%**	**1,56%**	**2,62%**	**2,62%**	**5,03%**

Mispredictions relative to TAGE-SC-L in % (lower better)

Increase in FIFO size leads to more mispredictions ⇒ only very recent outcomes are beneficial for stride

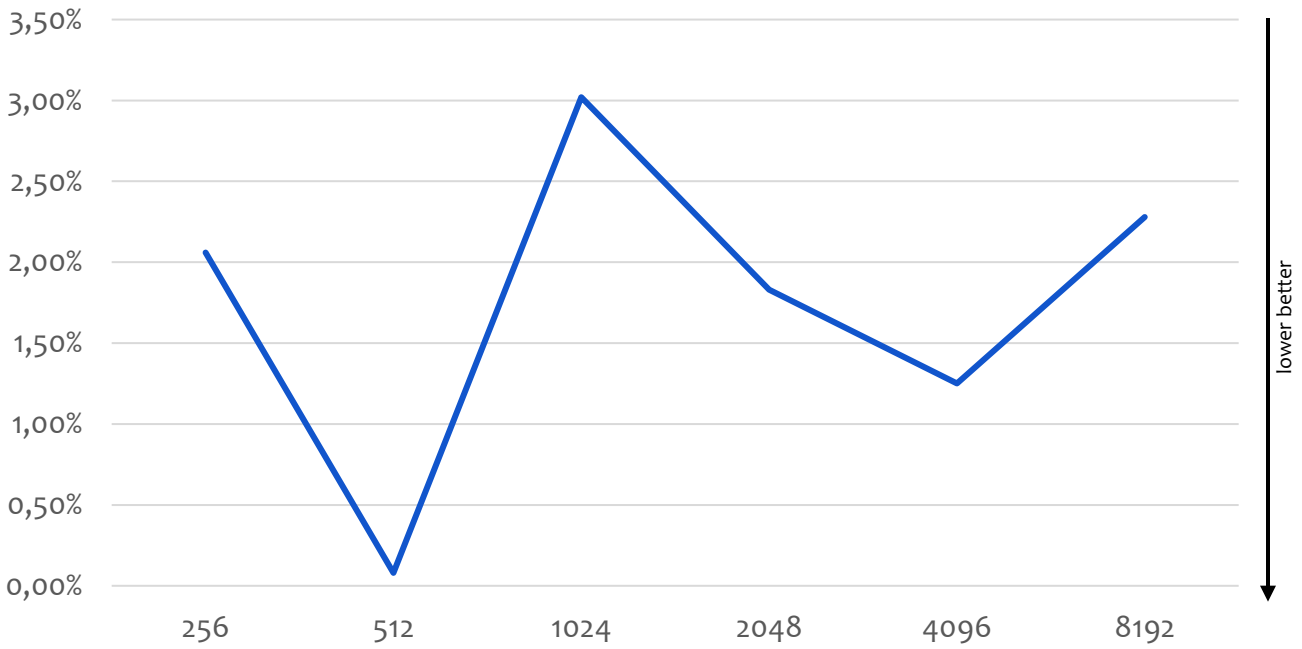
Evaluation: BRC Capacity – Bucket Count Training



Average mispredictions relative to uncapped BRC by bucket count in % (lower better)

An increase in Buckets leads to fewer mispredictions

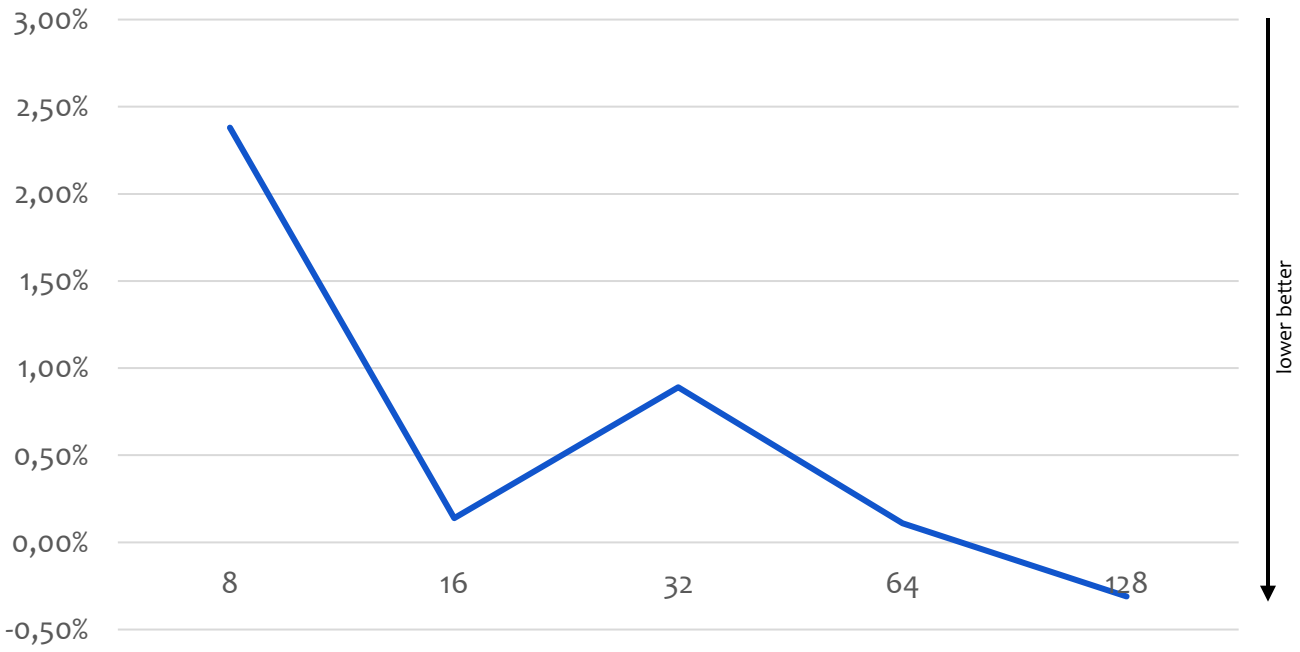
Evaluation: BRC Capacity – Bucket Count Stride



Average mispredictions relative to uncapped BRC by bucket count in % (lower better)

An increase in buckets does not lead to fewer mispredictions

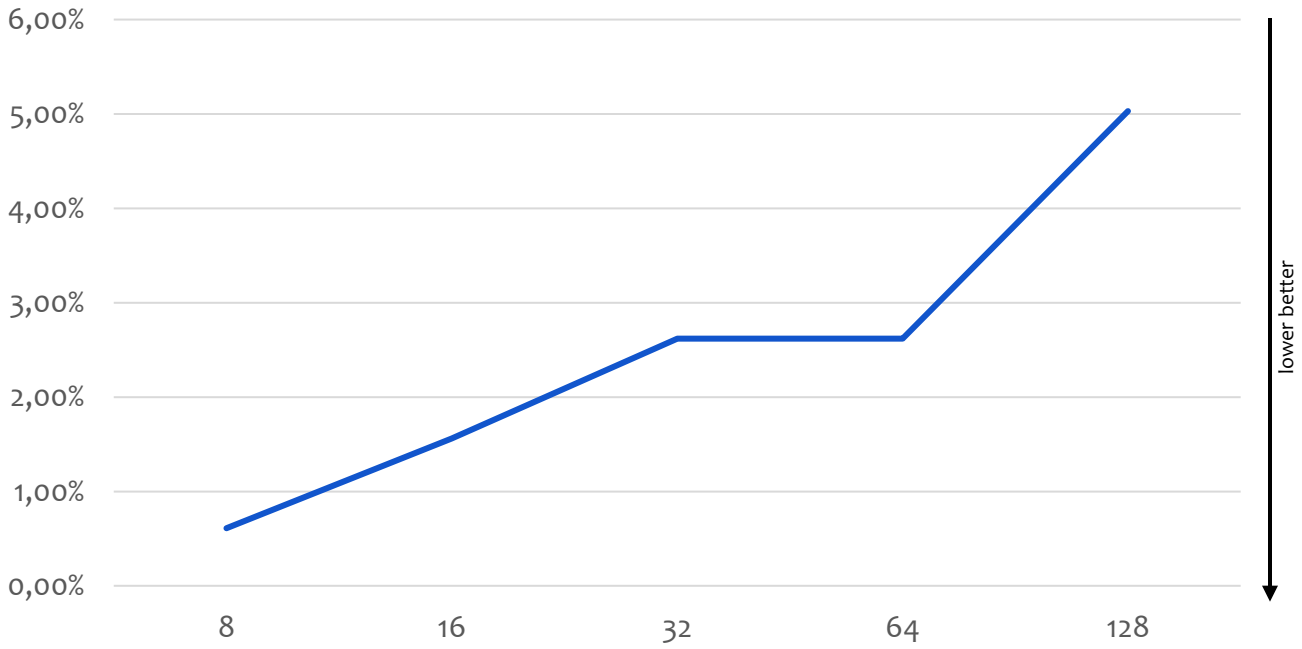
Evaluation: BRC Capacity – FIFO Size Training



Average mispredictions relative to uncapped BRC by FIFO size in % (lower better)

Increase in FIFO size slightly reduces mispredictions

Evaluation: BRC Capacity – FIFO Size Stride



Average mispredictions relative to uncapped BRC by FIFO size in % (lower better)

Increase in FIFO size leads to more mispredictions ⇒ only very recent outcomes are beneficial for stride

Evaluation: HTP-Branch Analysis Precision Control Mechanisms

Benchmark	Branch Count	HTP Branch Count	Training	Stride Detection
505.mcf	104	10	30.00%	40.00%
523.xalancbmk	1025	54	70.37%	62.96%
531.deepsjeng	889	206	83.98%	60.98%
541.leela	708	134	72.39%	45.64%
compression	129	13	23.08%	38.46%
dacapo-luindex	2929	23	82.61%	39.13%
dacapo-lusearch	5259	180	69.44%	50.00%
dacapo-spring	44265	10000	50.86%	45.64%
mediawiki	28651	2000	76.89%	56.03%
nodeapp	65543	26000	55.52%	52.42%
AVERAGE			**62%**	**49.13%**

Evaluation of HTP branches

HTP-branches are branches that contribute to 90% of mispredictions

How many of the HTP branches benefit from recycling

Both mechanisms achieve low rate in identifying beneficial branches

Evaluation: HTP-Branch Analysis Potential

Benchmark	Training	Potential Training	Stride	Potential Stride
505.mcf	-5.51%	0.80%	-0.31%	0.63%
523.xalancbmk	30.32%	34.50%	3.70%	17.43%
531.deepsjeng	28.49%	29.89%	2.83%	9.23%
541.leela	11.05%	13.69%	0.15%	6.90%
compression	-7.22%	1.37%	-0.63%	0.35%
dacapo-luindex	27.90%	38.56%	-9.20%	5.76%
dacapo-lusearch	-5.60%	10.38%	0.44%	5.03%
dacapo-spring	12.78%	19.21%	0.15%	6.90%
mediawiki	20.43%	23.13%	0.77%	7.49%
nodeapp	22.18%	27.25%	13.81%	20.48%
AVERAGE	**13.48%**	**19.88%**	**1.17%**	**8.02%**

Mispredictions relative to TAGE-SC-L of HTP - branches in % (higher better)

Both mechanisms achieve low rate in identifying beneficial branches

Sources:

1. Chit Kwan Lin and Stephen J. Tarsa Branch: Prediction Is Not a Solved Problem: Measurements, Opportunities, and Future Directions
2. Toward virtualizing branch direction prediction - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/The-TAGE-Predictor-with-N-Tagged-Tables-organization-of-the-TAGE-predictor-It-consists_fig1_254023355 [accessed 26 Oct 2025]
3. Seznec, André & Michaud, Pierre. (2006). A case for (partially) TAgged GEometric history length branch prediction. Journal of Instruction-level Parallelism - JILP. 8.
4. SPEC CPU 2017: https://www.spec.org/cpu2017/Docs/benchmarks/505.mcf_r.html
5. David Schall svr-bench: https://github.com/dhschall/gem5-svr-bench/tree/main/benchmarks
6. Fleetbench: https://github.com/google/fleetbench